



# SSAFYSTAR-K 싸스케

- 실시간 화상 노래방 서비스 -

---

## 포팅 매뉴얼

# 목차

|                        |    |
|------------------------|----|
| <b>I. 개요</b>           | 3  |
| 1. 프로젝트 개요             | 3  |
| 2. 프로젝트 사용 도구          | 3  |
| 3. 개발환경                | 4  |
| 4. 외부 서비스              | 5  |
| 5. Gitignore 처리한 핵심 키들 | 5  |
| <b>II. 빌드 및 배포</b>     | 6  |
| 1. 환경변수 형태             | 6  |
| 2. 빌드하기                | 8  |
| 3. 배포하기                | 8  |
| 4. 외부 서비스 설정 방법        | 10 |

## I. 개요

### 1. 프로젝트 개요

*"노래방 가고 싶은데 다 같이 모이기는 어렵고, 혼자 부르자니 심심한데?"*

*"이왕 부르는 거, 친구들이랑 점수로 한판 겨뤄보고 싶지 않으세요?"*

SSAFYSTAR-K(이하 서비스)는 이런 아쉬움에서 출발한 실시간 화상 노래방 서비스입니다. 어디에 있는 초대 코드 하나면 친구들과 같은 방에 모여, 얼굴을 보며 노래할 수 있습니다.

초대 코드로 입장한 최대 4명의 참가자가 카메라와 마이크를 공유하며 MR에 맞춰 노래를 부르고, 실시간 채팅으로 소통하며, 채점 결과를 리더보드로 거릅니다. 여기에 두 가지 모드로 즐길 거리를 더했습니다.

### 2. 프로젝트 사용 도구

#### ■ 이슈 및 형상 관리

Jira, GitLab

#### ■ 커뮤니케이션

Mattermost, Notion, Discord

#### ■ 디자인

Figma, Google Stitch

#### ■ CI/CD

GitLab Runner, Jenkins

### 3. 개발환경

#### ■ 프론트엔드 런타임 / 도구

VS Code : 1.131.0

Node.js : 24.18.0

pnpm : 9.10.0

Next.js : 16.2.10 (App Router, Turbopack)

TypeScript : 5.9.3

React : 19.2.4

#### ■ 백엔드 런타임 / 도구

IntelliJ IDEA : 2025.2.1 (Ultimate Edition)

JVM : JDK 17 (Eclipse Temurin 17)

Spring Boot : 4.1.0

Gradle : 9.5.1

#### ■ AI 런타임 / 도구

VSCode : 1.132.0

Python : 3.11

FastAPI : 0.141.1

Uvicorn : 0.52.1

PyTorch : 2.5.1+ cu121 (CUDA 12.1)

#### ■ 데이터 / 저장소

MySQL : 8.0

Redis : 7

PostgreSQL : 18.4

PGVector : 0.8.6

## 4. 외부 서비스

아래는 본 서비스가 연동하는 외부 서비스입니다.

- AWS S3 : MR·음원·프로필 이미지 등 정적 파일 저장 (버킷 + IAM 액세스 키 필요)
- AWS ELB : HTTPS 처리, 경로 기반 라우팅, WebSocket 프록시, ACM 인증서 연결
- AWS Certificate Manager : HTTPS (SSL) 인증서 발급 후 ALB 에 연결
- AWS EC2 : 백엔드 / 미디어 서버 호스팅 (Docker 컨테이너 기반)
- Vercel : 프론트엔드 서버 호스팅
- Railway : AI 서버 호스팅
- RunPod : AI GPU 추론
- OpenVidu : 실시간 화상/음성 미디어 서버
- Gambia : 도메인 등록 및 DNS 관리
- OAuth 로그인 : Google / Kakao (각 제공자 별 클라이언트 ID/Secret 필요)

## 5. Gitignore 처리한 핵심 키들

Next.js : .env.local (/frontend)

Spring Boot : application-secret.yml (/backend/.../src/main/resources)

FastAPI : .env (/ai/scoring-feedback)

## II. 빌드 및 배포

### 1. 환경변수 형태

Next.js (.env.local)

```
# API 서버 엔드포인트
NEXT_PUBLIC_API_URL=CHANGE_ME
API_URL=CHANGE_ME

# AI 서버 엔드포인트
NEXT_PUBLIC_AI_SERVER_URL=CHANGE_ME

# 미디어 서버 엔드포인트
NEXT_PUBLIC_OPENVIDU_URL=CHANGE_ME

# Vercel 배포 시 CLI가 자동 생성하는 토큰
VERCEL_OIDC_TOKEN=CHANGE_ME
```

Spring Boot (application-secret.yml)

```
# AWS S3 접근 정보
aws:
  credentials:
    access-key: CHANGE_ME
    secret-key: CHANGE_ME

# OpenVidu 미디어 서버 설정
openvidu:
  url: CHANGE_ME
  secret: CHANGE_ME
  webhook-secret: CHANGE_ME

# Google OAuth 로그인 설정
google:
  client-id: CHANGE_ME
  client-secret: CHANGE_ME

# Kakao OAuth 로그인 설정
kakao:
  client-id: CHANGE_ME
  client-secret: CHANGE_ME

# JWT 토큰 서명 키
jwt:
  secret: CHANGE_ME
```

```
# AI 서버 내부 통신 및 업로드 티켓 서명 키
```

```
ai:
```

```
    internal-api-key: CHANGE_ME
```

```
    upload-ticket-secret: CHANGE_ME
```

```
# MySQL 연결 정보
```

```
db:
```

```
    url: CHANGE_ME
```

```
    username: CHANGE_ME
```

```
    password: CHANGE_ME
```

```
# Redis 연결 정보
```

```
redis:
```

```
    host: CHANGE_ME
```

```
    password: CHANGE_ME
```

## FastAPI (.env)

```
# 서버 실행 설정
```

```
PORT=8000
```

```
CORS_ORIGINS=CHANGE_ME
```

```
# GMS / LLM 설정
```

```
GMS_KEY=CHANGE_ME
```

```
GMS_BASE_URL=https://gms.ssafy.io/gmsapi/api.openai.com/v1
```

```
GMS_MODEL=gpt-5-mini
```

```
GMS_REASONING_EFFORT=minimal
```

```
# STT 설정
```

```
# WHISPER_API_KEY를 비워 두면 GMS_KEY를 사용
```

```
WHISPER_API_KEY=CHANGE_ME
```

```
WHISPER_API_URL=https://gms.ssafy.io/gmsapi/api.openai.com/v1/audio/transcriptions
```

```
WHISPER_API_MODEL=whisper-1
```

```
WHISPER_API_TIMEOUT_SECONDS=120
```

```
# PostgreSQL / pgvector RAG 설정
```

```
RAG_DATABASE_URL=CHANGE_ME
```

```
RAG_COLLECTION_NAME=vocal_feedback_v1
```

```
GMS_EMBEDDING_MODEL=text-embedding-3-small
```

```
RAG_EMBEDDING_DIMENSIONS=1536
```

```
# Spring Boot 내부 API 연동 설정
```

```
SPRING_API_BASE_URL=CHANGE_ME
```

```
AI_INTERNAL_API_KEY=CHANGE_ME
```

```
SPRING_API_TIMEOUT_SECONDS=10
```

## 2. 빌드하기

### 1) Next.js

```
pnpm install
pnpm build
```

### 2) Spring Boot

```
./gradlew clean bootJar
```

### 3) FastAPI

```
pip install -r requirements.txt
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

## 3. 배포하기

본 서비스는 프론트엔드, 백엔드/미디어, AI 서버가 분산된 구조입니다.

이 중 백엔드 API와 STOMP(WebSocket) 트래픽을 ALB가 담당하며, HTTPS 처리 / 경로 기반 라우팅 / WebSocket 프록시를 수행합니다.

(프론트엔드의 경우 Vercel이 자체 SSL/CDN으로 서빙하므로 ALB 뒤에 있지 않습니다.)

### [ 사전 준비 ]

- 백엔드가 Docker 컨테이너로 구동 중인 EC2 인스턴스
- Gambia 에 등록된 도메인 및 DNS 관리 권한
- ACM 에서 API 도메인용 SSL 인증서 발급
- OpenVidu 서버는 별도 호스트에서 이미 구동 중

### [ 1단계 ] 대상 그룹 (Target Group) 생성

- API 서버 TG : HTTP (80)
- WebSocket 서버 TG : HTTP (80), 8080
- 각 TG 의 상태 검사 경로 지정 (예: /actuator/health)
- WebSocket TG 는 고정 세션 활성화 (같은 클라이언트가 항상 동일 인스턴스로 라우팅)

#### [ 2단계 ] Application Load Balancer 생성

- 유형 : Application Load Balancer
- 가용 영역(AZ) 2 개 이상 선택
- 보안 그룹 : 인바운드 80, 443 허용 (0.0.0.0/0 또는 필요 범위)

#### [ 3단계 ] 리스너 및 규칙 구성

- HTTP (80) → HTTPS (443) 리다이렉트
- HTTPS 리스너에 ACM 인증서 연결
- 아래와 같이 경로 기반 규칙을 추가

| 리스너 / 경로              | 동작(Action)              | 대상 그룹(Target Group) |
|-----------------------|-------------------------|---------------------|
| HTTP :80              | HTTPS :443 로 리다이렉트(301) | -                   |
| HTTPS :443 /api/*     | forward                 | API 서버 TG           |
| HTTPS :443 /ws-stomp* | forward (Stickiness ON) | WebSocket 서버 TG     |

#### [ 4단계 ] WebSocket / 유휴 시간 (Idle timeout)

- ALB 는 WebSocket 업그레이드를 기본 지원
- ALB 속성에서 유휴 제한 시간을 충분히 상향 (기본 60 초)

#### [ 5단계 ] 실시간 미디어 (WebRTC) - OpenVidu

- 화상/음성 미디어는 UDP/SRTP 라 HTTP(S)용 ALB 로는 프록시 되지 않음
- ALB 를 경유하는 것은 방/참가자 동기화 및 채팅용 STOMP 시그널링 뿐
- 미디어 스트림은 클라이언트가 OpenVidu 서버와 직접 연결

#### [ 6단계 ] DNS 연결 (Gabia)

- 프론트엔드 : Gabia 에서 도메인을 Vercel 이 안내하는 레코드로 연결
- 백엔드 : Gabia 에서 도메인을 ALB DNS 이름으로 CNAME 연결

- Gabia 는 AWS Route 53 이 아니므로, ALB 연결에 A 레코드를 사용할 수 없음

#### [ 7단계 ] 배포 확인

- 대상 그룹의 대상 상태가 모두 'healthy' 인지 확인
- 적용한 도메인 접속 확인
- 로그인 → 방 생성 → 화상/음성 → 채팅 → 채점까지 전 기능 동작 확인

## 4. 외부 서비스 설정 방법

아래는 본 서비스가 사용하는 외부 서비스의 설정 절차입니다.

### 가) AWS S3 (음원·정적 파일 저장)

1. S3 콘솔에서 버킷 생성 (버킷 이름은 백엔드 저장 위치로 사용됨)
2. IAM > 사용자/역할 생성 후 S3 접근 권한(AmazonS3FullAccess 또는 최소 권한) 부여
3. 액세스 키 발급 → 백엔드 설정(application-secret.yml)에 등록

### 나) ACM / Gabia DNS (HTTPS·도메인)

1. ACM에서 도메인 인증서 요청 → DNS 검증용 CNAME 레코드를 Gabia에 등록 후 발급
2. 발급된 인증서를 ALB의 HTTPS:443 리스너에 연결
3. Gabia에서 등록한 도메인을 ALB DNS 이름으로 CNAME

### 다) OpenVidu (실시간 화상/음성)

1. OpenVidu 서버 구동 확인 및 접속 URL / Secret 확보
2. 백엔드 설정에 OPENVIDU\_URL / OPENVIDU\_SECRET / Webhook Secret 등록
3. 프론트 .env-local의 NEXT\_PUBLIC\_OPENVIDU\_URL에 접속 URL 등록

### 라) OAuth 로그인 (예: Google / Kakao)

1. 각 개발자 콘솔에서 애플리케이션 등록 → 클라이언트 ID/Secret 발급
2. 리디렉션 URI에 배포 도메인 등록 (예: https://[도메인]/login/oauth2/code/[provider])
3. 발급 키를 백엔드 설정에 등록